Received 23 January 2025; revised 17 February 2025; accepted 2 March 2025. Date of publication 7 March 2025;
date of current version 16 April 2025. The review of this article was arranged by Associate Editor Prof. Rosdiadee Nordin.

Digital Object Identifier 10.1109/SR.2025.3548547

Tiny Federated Learning for Constrained Sensors: A Systematic Literature Review

CLAUDIO JR. NASCIMENTO DA SILVA AND CÁSSIO V. S. PRAZERES (Senior Member, IEEE)

Institute of Computing, Federal University of Bahia, Salvador 40170-110, Brazil

CORRESPONDING AUTHOR: CLAUDIO JR. NASCIMENTO DA SILVA (email: claudiojns@ufba.br)

This work was supported in part by FAPESB INCITE under Grant PIE0002/2022, in part by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) through the CAPES Transformative Agreement with IEEE, and in part by the CNPq, CAPES, and FAPESB organizations for supporting the Graduate Program in Computer Science at the Federal University of Bahia. The work of Claudio Junior Nascimento da Silva was supported by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) through doctoral scholarship under Grant 160952/2021-2.

ABSTRACT The recent tiny federated learning (TinyFL) paradigm has generated significant interest in enabling resource-constrained edge devices to perform local machine learning training. TinyFL reduces latency, communication costs, and privacy risks by sharing model weights or gradients instead of raw data. Our research provides a systematic literature review (SLR) of 30 selected studies to understand the current research landscape in TinyFL. The review highlights that most studies focus on Internet of Things (IoT) environments, using lightweight architectures and optimization techniques to overcome challenges related to limited memory, processing power, and energy resources. While most studies address generic IoT applications, emerging efforts in domains, such as the Internet of Medical Things and unmanned aerial vehicles, highlight the adaptability of TinyFL to diverse scenarios. Common strategies include using FedAvg to aggregate learning models, such as multilayer perceptron, convolutional neural networks, and deep neural networks. Frameworks, such as TensorFlow Lite Micro and Python, enable local training on a diverse range of devices, including microcontrollers, such as the ESP32 and Arduino Nano BLE 33, as well as more powerful single board, such as the Raspberry Pi 4 Model B. The limitation of data storage on devices and the security risks inherent in sharing models are gaps identified in the analysis of the studies. In addition to providing a current overview of existing research, this SLR contributes to the advancement and progress of the TinyFL paradigm by providing insights into its challenges, strategies, and future directions.

INDEX TERMS Internet of Things (IoT), tiny devices, tiny federated learning (TinyFL).

I. INTRODUCTION

The spread of the Internet over the last decade has brought some changes to the daily lives of individuals, companies, and society as a whole. This phenomenon boosted the development of technologies, such as the Internet of Things (IoT), which was leveraged by popularization, miniaturization of devices, and cost reduction. Society, in general, has started to adapt to the presence of sensors and actuators, which help in various activities and, in many cases, improve the quality of life. Another area that has developed significantly in recent years is machine learning (ML), which has started to assist in executing repetitive tasks and, especially, in solving complex problems.

IoT and ML are two emerging technologies that have attracted significant interest from researchers in recent years. Integrating the IoT and ML has become an important research topic in networking, as it uses sensors to gather data for ML model execution on a central server. However, challenges, such as bandwidth limitation, latency constraints, privacy concerns, and data silos, hinder complete integration [1]. To address these issues, fog/edge computing and federated learning (FL) bring intelligence closer to the edge and consequently to the end user, reducing latency and bandwidth needs [2].

The tiny machine learning (TinyML) framework is an additional component in advancing IoT and ML integration. First introduced by Warden and Situnayake [3] in their foundational work, TinyML enables ML models to operate on resource-limited edge devices, such as microcontrollers, which face substantial processing, memory, and power consumption constraints. The literature highlights multiple successful

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

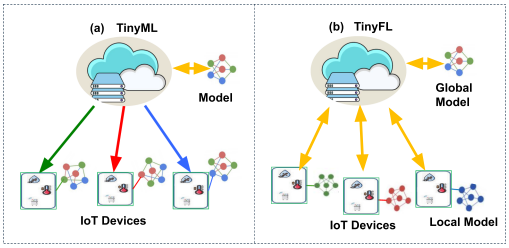


FIGURE 1. TinyML × TinyFL.

applications of TinyML across various domains, showcasing its utility in effectively inferring ML models. Nonetheless, challenges persist, particularly regarding efficiency and model updating [4], [5].

Researchers are expanding TinyML's capabilities through integrating FL, resulting in tiny federated learning (TinyFL) [6], [7], [8]. In this approach, resource-constrained devices train models locally and collaborate by sharing model updates, as in traditional FL. Fig. 1(a) illustrates the TinyML model, where ML models run directly on edge devices, and Fig. 1(b) illustrates the TinyML approach, which adds local training on these devices. In TinyFL, model weights are periodically sent to a central server for aggregation, disseminating an updated global model to all participating clients.

The TinyFL framework increases the demands of on-device processing, memory, and energy as devices perform inferences and train models locally, transmit them to a server for aggregation, and receive an updated global model [9]. Research on TinyFL remains in its early stages. Wulfert et al. [10] and Muhammed and Ibraheem [11] highlighted the challenges of implementing FL on resource-limited IoT devices, while Ficco et al. [12] discussed specific hardware configurations for these experiments. Giménez et al. [13] addressed the nuances of local training on constrained devices, with most current publications focusing on isolated issues within FL for limited-resource environments. However, none present a complete solution that runs the entire FL cycle, evaluates the results, and keeps the device collecting data.

This study aims to explore the state-of-the-art research on TinyFL through a systematic literature review (SLR). Based on the academic literature, we applied the SLR method to extract and synthesize information from IoT solutions that approach TinyML and FL. The study focuses on solution design, system architectures, environments, and devices. The results reflect the actuality of IoT solutions using the TinyFL paradigm. In summary, the contributions of this study include the following.

1) Identify the motivations for using the TinyFL paradigm in IoT solutions.
2) Identify the main challenges of applying FL to resource-constrained devices.
3) Provide insight into the current state of research at TinyFL.

TABLE 1. Analysis of Related Work

Authors	TinyFL	Local Training	Specifies Devices	Specifies Frameworks
Ray [14]	No	No	No	Partially
Dutta and Bharali [15]	No	No	No	No
Oliveira et al. [16]	No	No	No	No
Tekin et al. [17]	Yes	Yes	No	Partially
Qi et al. [18]	Yes	Yes	Partially	Yes
Fragkou and Katsaros [19]	Yes	Yes	Partially	Yes
This study	**Yes**	**Yes**	**Yes**	**Yes**

To the best of the authors' knowledge, there are no published systematic reviews on TinyFL, and this research may be one of the first attempts to consolidate knowledge about this emerging area. Unlike other reviews that address FL in general, our study focuses explicitly on the challenges and advances of FL on resource-constrained devices.

The rest of this article is organized as follows. Section II presents the related work. Section III describes the research method we used to select and analyze the studies, and Section IV presents the search results. Section V discusses the answers to the motivations for integrating the FL paradigm in IoT solutions and presents the results that we obtained in our systematic review. Section VI outlines the open challenges in the field. Finally, Section VII concludes this article.

II. RELATED WORKS

Table 1 compares our work with related studies, focusing on integrating TinyML and FL (TinyFL) and local training on edge devices. Although works, such as Ray [14], Dutta and Bharali [15], and Oliveira et al. [16], focus only on TinyML or general local inference concepts, our study stands out by addressing issues of energy consumption, privacy, and security. Furthermore, unlike other works, our study specifies the devices used and details the frameworks and software that support IoT applications, filling critical gaps in the field.

Ray [14] provided an overview of TinyML, addressing its fundamentals, challenges, tools, and applications. The paper discusses general aspects of hardware, software, and libraries; it does not specify particular devices or implementation details. The paper offers a broad perspective on TinyML but lacks an in-depth exploration of real-world applications or the complexities of implementing TinyML models in specific scenarios.

Dutta and Bharali [15] provided an overview of the intersection of TinyML and IoT, focusing on how lightweight models can enhance IoT applications. The paper discusses the integration of TinyML with emerging technologies, such as 5G, but does not specify particular devices or detailed implementations. While it highlights general software and hardware considerations, it lacks explicit mentions of frameworks or

toolsets. The study is more conceptual, addressing the potential impact of TinyML rather than practical applications or implementation specifics.

Oliveira et al. [16] explored the use of TinyML in smart cities and agriculture, highlighting its potential to optimize resource use and decision-making in these domains. However, the paper remains theoretical, focusing on the high-level benefits of TinyML without detailing specific devices or hardware implementations. Similarly, it does not specify frameworks but discusses TinyML's general applicability. The study provides insight into possible applications but lacks concrete technical depth for real-world deployment.

Tekin et al. [17] examined energy-efficient TinyML solutions, focusing on lightweight models and optimization strategies. The paper highlights model pruning and quantization methods to adapt TinyML to resource-constrained environments. Although it discusses devices in general terms, it does not provide detailed specifications or examples of particular hardware. The paper partially addresses frameworks, mentioning popular tools, such as TensorFlow Lite, but without comprehensive technical details or evaluations of their effectiveness.

Qi et al. [18] delved into lightweight FL (TinyFL), exploring the adaptation of FL techniques to devices with severe resource constraints. The study includes discussions on hardware and software considerations, partially specifying devices, such as Raspberry Pi and microcontrollers. It also details frameworks, such as TensorFlow Lite and other tools, optimized for TinyML applications. The paper balances theoretical insights with practical considerations, making it more detailed than general overviews.

Fragkou and Katsaros [19] provided a joint survey on decentralized, FL and TinyML, explicitly focusing on swarm learning. The paper emphasizes training models locally on edge devices and discusses energy efficiency and privacy challenges. While it mentions general device categories, such as Cortex-M processors, it does not detail specific hardware implementations. Frameworks are addressed comprehensively, referencing tools, such as TensorFlow Lite, and their suitability for decentralized learning.

In conclusion, this section highlights our study's unique contributions by comparing it with previous research. To the best of the authors' knowledge, no existing studies specifically address FL in resource-constrained IoT sensors, further emphasizing the novelty of our approach.

III. RESEARCH METHOD

To systematize the knowledge about motivations and the design of IoT solutions that approach the TinyFL paradigm, we applied an SLR. An SLR is a study that aims to identify, evaluate, interpret, and synthesize all available research relevant to a particular question, a topic area, or a phenomenon of interest [20]. The selection of relevant studies was performed using Parsif.al software.[1] This section presents the method used to conduct this study, the research questions, the phases of the study, the study identification and search process, the study selection and eligibility criteria, and data extraction.

A. RESEARCH QUESTIONS

In this step, we present a set of research questions to guide the systematic exploration of TinyFL in IoT environments.

These questions address key aspects of the field, including the motivations behind its adoption, the architectures employed, the methods used to perform FL, the strategies for handling device constraints, the challenges faced, and the future directions of TinyFL research. To this end, we formulate the research questions (RQs) described below.

RQ1: *What is the focus of current research in TinyFL?* To understand the primary goals and motivations driving the adoption of FL in IoT systems with resource-constrained devices.

RQ2: *In which domains have the selected studies conducted their research on TinyFL applications?* To identify the domains in which the selected studies conducted their research provides essential context for understanding the practical applications and relevance of TinyFL.

RQ3: *What are the architectural and methodological choices in TinyFL implementations?* To understand how TinyFL is implemented, it is essential to explore key architectural decisions, training locations, aggregation strategies, and ML algorithms, as these directly impact efficiency and applicability in resource-constrained environments.

RQ4: *What devices are used in TinyFL implementations, and how do they address resource constraints?* To analyze strategies for overcoming limitations, such as low memory, processing power, and energy in tiny devices.

RQ5: *What are the challenges and future directions for TinyFL?* To identify obstacles in implementing TinyFL, provide insights into areas requiring further research or innovation, predict trends, highlight potential advancements, and uncover unexplored opportunities in the field.

This study seeks to provide a comprehensive understanding of the state of the art and identify opportunities for further innovation in this domain by answering these questions.

B. PHASES OF THE STUDY

The SLR process is organized into four phases according to the guideline proposed by Kitchenham and Charters [21].

1) Identifying potential studies by querying credible sources from our research questions.
2) Selecting studies based on their relevance and quality to the research questions.
3) Extracting data from the chosen articles according to research questions.
4) Synthesizing and analyzing the data extracted to answer the research questions.

[1] [Online]. Available: https://parsif.al/

TABLE 2. Research Databases

Name	URL
ACM Digital Library	https://dl.acm.org
IEEE Xplore	https://ieeexplore.ieee.org
MDPI	https://www.mdpi.com
Science@Direct	https://www.sciencedirect.com
Scopus	https://www.scopus.com
Wiley Online Library	https://onlinelibrary.wiley.com
arXiv	https://arxiv.org

TABLE 3. Number of Studies by Database

Database	Number of Studies	Duplicated	Excluded	Selected
ACM Digital Library	2	0	1	1
IEEE Xplore	74	7	53	14
MDPI	8	0	6	2
Science@Direct	39	0	37	2
Scopus	147	50	87	10
Wiley Online Library	0	0	0	0
arXiv	1	0	0	1
Total	**271**	**57**	**184**	**30**

C. STUDY IDENTIFICATION AND SEARCH PROCESS

The identification of the study is the first step in this SLR. We identified potential studies from six bibliographic databases: ACM Digital Library, IEEE Xplore, MDPI, Science@Direct, Scopus, Wiley on Line Library, and arXiv outlined in Table 2.

The search process was built following a structured query to retrieve potential studies from the chosen databases. The query comprises multiple keywords combined using logical operators to capture a broad range of studies matching the specified metadata. Keywords were connected using `AND` and `OR` to form a comprehensive search string. The final search string used is as follows: *(tinyfl OR tiny-fl OR "tiny fl" OR "TinyFL" OR "tinyfed∗" OR "fedtiny∗") OR ["FL" AND ("microcontroller∗" OR "constrained-device∗" OR "resource-constrained device∗" OR "low-power device∗" OR "memory-constrained device∗")]* . First, we identified potential studies from databases. This step resulted in 30 studies outlined in Table 3.

D. STUDY SELECTION AND ELIGIBILITY CRITERIA

The initial set of studies encompasses irrelevant results. To effectively address the research questions, it is necessary to discard these irrelevant entries and ensure the inclusion of suitable studies. To this end, we have established the following inclusion (I) and exclusion (E) criteria.

1) (I1) Include studies that address specific improvement objectives or technical problems of IoT systems with TinyFL.
2) (E1) Exclude patents, books, preprints, technical reports, white studies, short studies, and position studies.
3) (E2) Exclude studies whose title and abstract are unrelated to TinyFL.
4) (E3) The study should present initiatives related to the topic of the research strings.
5) (E4) The selected study must be available online.
6) (E5) The study must be unique. Duplicate entries need to be removed.
7) (E6) Exclude studies not written in English.

Second, we applied an initial selection using the criteria above, followed by a detailed analysis of the full text of the studies. After removing duplicates, we identified 212 unique records for further evaluation. Applying the inclusion and exclusion criteria during the full-text review excluded 184 articles, leaving 30 studies used for data synthesis. Table 3 presents the distribution of studies across the evaluated databases, showing the number of duplicates, excluded, and selected studies.

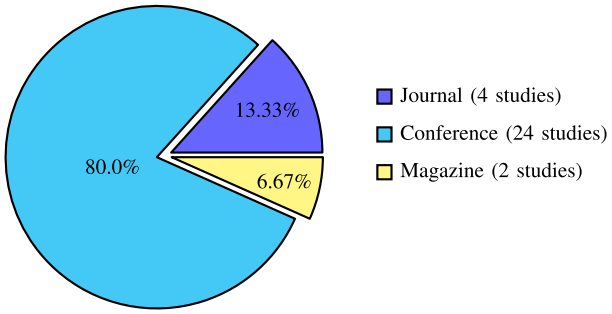


FIGURE 2. Document type.

E. DATA EXTRACTION AND DATA SYNTHESIS

We extracted the data required to answer the research question from the selected studies and stored the data in a tabular format. We extracted the following data for each research question mentioned in the studies.

IV. SEARCH RESULTS

Of the 30 studies selected in the SLR, 24 (80.00%) are conference proceedings, four (13.33%) are journal articles, and two (7.14%) are magazine articles. All 30 are research articles; no review articles, book sections, books, reports, or working studies are included. Fig. 2 presents the distribution of document types. We mainly highlight the absence of review articles, which underscores the pioneering nature of our work in this domain.

Fig. 3 illustrates the distribution of selected studies by year of publication after the study's second phase. Most studies were published in the year 2023, accounting for 46.67% of the total (14). The first study on TinyFL appeared in the year 2020, with one study (3.33%), and in the year 2021, with two studies (6.67%). In the year 2022, the number of studies remained consistent at (6.67%) (two studies). In the year 2024, there was a significant contribution, with 11 studies published, representing 36.67% of the total.

The distribution of studies per year demonstrates that TinyFL is an emerging field with significant growth starting

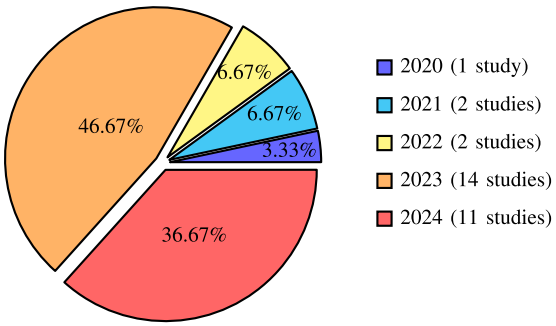


FIGURE 3. Studies per year.

in 2023. The continued number of publications in 2024 suggests that the field is consolidating and attracting researchers, especially as challenges related to efficiency, privacy, and integration with edge devices become increasingly relevant.

V. DATA SYNTHESIS RESULTS

In the previous section, we analyzed the evolution of publications over time, providing a broader perspective on the growth of research in TinyFL. This section addresses the research questions by thoroughly examining the selected studies. Table 4 summarizes the key contributions and publication years of the 30 studies accepted in our review. These studies reflect the diversity and progression of research efforts, offering valuable insights into the current state of TinyFL.

A. RESEARCH FOCUS AND PROGRESS IN TINYFL

This section addresses the research question *RQ1* by analyzing the focus of current studies in TinyFL. The selected articles provide insights into various aspects of TinyFL, such as optimizing memory usage (e.g., FedTinyWolf), enhancing communication efficiency (e.g., TinyFL), adapting meta-learning algorithms for resource-constrained devices (e.g., TinyReptile), and implementing FL directly on Microcontroller Unit (MCU) (e.g., TinyFedTL). This analysis underscores the breadth and diversity of current research efforts in the field.

Wulfert et al. [10] and Muhammed and Ibraheem [11] examined the challenges associated with deploying FL on IoT devices with limited computational resources. Key issues identified include security and privacy risks related to central server use, complexities in the random selection of network links for point-to-point communication, and mechanisms for protecting user privacy. Wulfert et al. [10] proposed using gossip learning to address decentralized communication issues in FL. Similarly, Muhammed and Ibraheem [11] introduced a novel algorithm, Alpha-FedAvg, although the implementation specifics remain undisclosed.

Huang et al. [22] developed a distributed pruning approach called FedTiny to create tiny NNs in FL environments.

Giménez et al. [13], [23] explored FL on microcontrollers, finding that more frequent training rounds lead to faster reductions in training loss, although this comes with increased training time and bandwidth usage. They further observed that models with additional hidden layers enhance loss reduction rates but require greater RAM capacity. In [33], they focus on bandwidth limitations in long range (LoRa) networks. The authors integrate LoRaMesher as a networking service within a distributed FL application to address this. Their open-source prototype demonstrates practical FL with efficient communication over LoRa networks.

Dai and Tan [24] proposed an FL method optimized for computational and communication efficiency on microcontrollers, emphasizing the difficulties of implementing FL on devices with limited processing power. Sriram et al. [25] explored the feasibility of deploying healthcare applications using FL on low-power MCUs, such as the Arduino Nano 33 BLE Sense. Their study addresses energy constraints and briefly touches on privacy preservation without an in-depth analysis of the latter aspect.

Adhikary and Dutta [7] introduced FedTinyWolf, a novel FL mechanism designed explicitly for microcontrollers with enhanced memory efficiency. FedTinyWolf leverages a modified grey wolf optimizer (GWO) and a multilevel feedback loop mechanism to optimize memory usage and improve model accuracy.

Aatab and Freitag [26] introduced an NN library specifically designed for integration within an FL framework to enhance the adaptability of IoT devices in distributed learning scenarios. The experiments on real IoT hardware demonstrate that the library can achieve high accuracy rates, highlighting its effectiveness in supporting FL on devices with limited resources.

The work presented by Das et al. [27] centers on enhancing the accuracy of federated models in environments with heterogeneous devices. The primary challenge addressed is the limited computational performance of devices with constrained resources. The authors report accuracy gains in scenarios involving restricted devices compared to conventional setups relying solely on low-capacity devices.

Xiong and Punnekkat [6] proposed a TinyFL algorithm (TFL-BC) designed to train Bayesian classifiers using distributed training data in an edge system. This approach efficiently leverages FL principles to accommodate resource-constrained devices while preserving data privacy and optimizing classifier performance.

Kopparapu et al. [28] introduced TinyFedTL, the first attempt to implement federated transfer learning (FTL) at the most constrained MCU level in IoT environments. TinyFedTL enables even resource-limited wearables and patient sensors to continuously assess situations and learn without risking privacy violations.

Cheng et al. [9] presented TinyFL, a lightweight FL method designed for efficient communication and memory usage for resource-constrained environments.

Wulfert et al. [29] introduced TinyFL, a novel approach to perform FL exclusively on MCUs, eliminating the need for personal computers as servers for communication and aggregation.

TABLE 4. Selected Studies

Authors	Title	Contribution	Year
Wulfert et al. [10]	Adaptive decentralized federated gossip learning for resource-constrained IoT devices	Presents GossipFL and AdaLoss, approaches that aim to optimize model delivery and reduce communication costs in IoT networks with limited resources	2023
Muhammed and Ibraheem [11]	AlphaFedAvg - Safeguarding privacy and enhancing forensic analysis in FL on edge devices	Proposes AlphaFedAvg, a model that uses dynamic parameter adjustment to improve flexibility and learning performance in different FL	2024
Huang et al. [22]	Distributed pruning toward tiny NNs in FL	Presents FedTiny, a distributed pruning approach with adaptive batch normalization selection for unbiased pruned models	2023
Giménez et al. [13]	Embedded FL over a LoRa mesh network	Proposes a decentralized architecture that divides ML and network tasks into two boards (Arduino P. H7 and TTGO-LORA), facilitating updates and parallel processing.	2023
Giménez et al. [23]	On-device training of ML models on microcontrollers with FL	Presents a prototype and analysis of on-device training of a keyword spotting model with FL on Arduino Nano 33 BLE Sense boards.	2022
Dai and Tan [24]	FedCFC: On-device personalized FL with closed-form continuous-time NNs	Develops FedCFC, a custom FL approach that exploits the involatility of a key module of Closed-Form Continuous-Time networks to federate only this module, reducing communication overhead.	2024
Sriram et al. [25]	FL on low-power Arduino Nano33 BLE sense to predict the length of stay using an LR model	Presents an FL framework to predict the length of stay of patients in hospitals, using an LR model on low-power devices	2024
Adhikary and Dutta [7]	FedTinyWolf—A memory efficient federated embedded learning mechanism	Introduces FedTinyWolf, a model that improves accuracy and energy efficiency compared to traditional FL models	2024
Aatab and Freitag [26]	Integrating an NN library in embedded FL	Uses the serial port of IoT boards for communication between clients and the server, avoiding the complexity of network protocols and eliminating network effects in experiments	2023
Das et al. [27]	Learning, tiny and huge: Heterogeneous model augmentation toward federated tiny learning	Introduces FedTinyAug, a method for coordinating devices with diverse capabilities, allowing everyone to participate in collaborative learning	2023
Xiong and Punnekkat [6]	TinyFLwith Bayesian classifiers	Introduces TFL-BC, an FL algorithm that uses Bayesian classifiers, adapting learning to distributed data and improving performance on resource-constrained devices	2023
Kopparapu et al. [28]	TinyFedTL: FTL on ubiquitous tiny IoT devices	Introduces TinyFedTL, a first attempt to perform FTL on MCU.	2022
Cheng et al. [9]	TinyFL: A lightweight FL method with efficient memory-and-communication	Proposes TinyFL, a sparse update scheme that reduces memory footprint and computational overhead by utilizing a parameter splitting method in the model upload process	2023
Wulfert et al. [29]	TinyFL: On-device training, communication and aggregation on a microcontroller for FL	Presents a master/slave method for communication between MCUs, where a master initializes and aggregates the models, and evaluates the time spent in each stage of the FL	2023
Ren et al. [30]	TinyReptile: TinyML with federated meta-learning	Formulates TinyReptile, an approach to finding good initial weights for NNs that can quickly learn new tasks with few examples	2023
Ren et al. [31]	TinyMetaFed: Efficient federated meta-learning for TinyML	Proposes TinyMetaFed, an efficient federated meta-learning algorithm for TinyML. It uses the Reptile meta-learning algorithm, making the process more efficient in communication and computation	2023
Zheng et al. [8]	Trust management of TinyFL in Internet of UAVs	Proposes a trust management model for tiny WFL that uses semicentralized blockchain, which considers direct and indirect trust values and offers advantages such as robustness, scalability and dynamic trust update	2024
Khoa et al. [32]	AOP toward AOP Approach in an FL framework for edge AI applications	Introduces Adaptive Offloading Point. This method uses reinforcement learning and Gaussian Mixture Model to determine which DNN layers should be offloaded to the server, accelerating training and improving efficiency on resource-constrained devices.	2023
Giménez et al. [33]	Demo Edge FL over a LoRa mesh network	Shows the feasibility of providing edge intelligence with adaptive nodes based on low-capacity, networked microcontrollers	2024
Wu et al. [34]	Efficient privacy-preserving FL for resource-constrained edge devices	Proposes a framework for training DNNs on edge devices, with privacy protection, reduced computing and communication costs, and reduced response latency by offloading computation to nearby edge servers.	2021
Hidayat et al. [35]	Efficient and secure privacy-preserving FL for resource-constrained devices	Develops a system that aims for privacy and efficiency in resource-limited environments, focusing on model accuracy and biases	2023
Gao et al. [36]	End-to-end evaluation of FL and split learning for Internet of Things	Performs comparative tests between FL and SplitNN, showing that the communication overhead of SplitNN is much higher than that of FL	2020
Akubathini et al. [37]	Evaluation of ML approaches for resource-constrained IIoT devices	Discusses suitable ML models and optimization methods for resource-constrained devices, as well as an evaluation of industrial FL for resource optimization on edge devices.	2021
Villegas-Ch et al. [38]	Optimizing FL on TinyML Devices for privacy protection and energye fficiency in IoT networks	Evaluates the impact of communication and workload on model accuracy and energy consumption, demonstrating that FL on TinyML devices can significantly reduce energy consumption without compromising model accuracy	2024
Mwawado et al. [39]	Optimizing soil-based crop recommendations with FL on Raspberry Pi edge computing nodes	Uses a public crop recommendation dataset to examine how FL parameters affect model accuracy, computational efficiency, and communication overhead on Raspberry Pi nodes	2024
Hidayat et al. [40]	Privacy-preserving FL with resource-adaptive compression for edge devices	Proposes a method that uses compression-based pruning to condense local gradients and determines the compression rate based on device resource availability, as well as incorporating adaptive clipping to control the variability of noise added to the model.	2024
Mpofu et al. [41]	A privacy-preserving FL architecture implementing data ownership and portability on edge end-points	Proposes an FL architecture based on P2P, low-cost IoT devices that allow data portability by owners, using General Data Protection Regulation principles.	2023
Nikić et al. [42]	Lightweight digit recognition in smart metering system using narrowband Internet of Things and FL	Develops a prototype digit reading system for smart meters, optimized for edge devices, with customized (CNN), image preprocessing and FL integration	2024
Colombo et al. [43]	TTIFeD: A tiny Integer-based FL algorithm with DFA	Proposes TIFeD, aninteger-based FL algorithm with DFA that is specifically designed to operate on resource-constrained devices, reducing memory and computation consumption	2024
Rimoli et al. [44]	TinyFL with blockchain for privacy and security preservation of MCU-based IoT applications.	Proposes a trust management model for TinyFL based on blockchain	2024

Ren et al. [31] proposed TinyMetaFed, which combines FL and meta-learning to enable rapid model adaptation in TinyML devices with limited computational resources. Utilizing the Reptile meta-learning algorithm, the method optimizes model weights to facilitate customization with few examples, reducing the need for full retraining. This approach improves energy efficiency, inference time, and adaptation to new tasks, making TinyFL more viable for IoT applications, such as environmental monitoring and wearable devices.

Ren et al. [30] presented TinyReptile, a lightweight meta-learning algorithm tailored for TinyML scenarios. TinyReptile enables resource-constrained IoT devices to collaboratively learn a robust NN initialization. It combines FL concepts and online learning to facilitate meta-learning on distributed IoT devices. It leverages the serial communication nature of FL and the Reptile meta-learning algorithm, enabling resource-efficient learning.

Zheng et al. [8] proposed a semicentralized trust management framework for blockchain-enabled Tiny FL wireless federated learning (WFL) in the Internet of unmanned aerial vehicles (UAV) systems. This framework delivers a reliable, fast, low-latency, and energy-efficient WFL aggregation model.

Khoa et al. [32] tackled the challenge of efficiently training artificial intelligence (AI) on IoT devices with limited resources. They proposed an adaptive offloading point (AOP) approach that uses reinforcement learning to determine which layers of deep neural networks (DNNs) should be offloaded to the server, thereby accelerating local training. The findings reveal a significant reduction in training time

compared to traditional methods, such as classical FL and FedAdapt.

Wu et al. [34] explored the difficulties of training deep learning models on resource-constrained edge devices while maintaining privacy and reducing costs. Their proposed system partitions model training between edge devices and nearby edge servers, leveraging differential privacy and lightweight encryption.

The studies by Hidayat et al. [35], [40] propose compression-based methods to enhance efficiency and privacy in resource-constrained edge devices. By optimizing gradient pruning and dynamically adjusting compression rates, these approaches achieve faster processing, reduced costs, improved accuracy, and greater resilience to attacks.

Gao et al. [36] compared FL and SplitNN in real-world IoT scenarios with non-independent and identically distributed (non-IID) and imbalanced data distributions. Implemented on Raspberry Pi devices, the study highlights that SplitNN achieves better learning performance on unbalanced datasets, while FL demonstrates lower communication overhead.

Akubathini et al. [37] examined ML algorithms suitable for Industrial Internet of Things (IIoT) devices, focusing on memory footprint and accuracy. Techniques, such as quantization, pruning, and FL, are employed to optimize model size and performance. The results demonstrate that these optimizations reduce computational costs while maintaining high accuracy.

Villegas-Ch et al. [38] explored the challenges of implementing FL on TinyML devices constrained by energy, communication, and processing. Their approach integrates FL with differential privacy and encryption techniques. The results show a 33% reduction in energy consumption compared to centralized methods, with only a 1.2% loss in accuracy.

Mwawado et al. [39] explored using FL to optimize soil-based crop recommendations on Raspberry Pi edge nodes. Their approach involves fine-tuning parameters, such as client count, training rounds, and local epochs, to enhance system efficiency. This method demonstrates significant potential for improving crop recommendation accuracy in edge computing environments.

Mpofu et al. [41] addressed the challenges of resource efficiency and privacy in edge devices. The authors propose a method that employs weight pruning-based compression to condense local gradients while dynamically adjusting the compression rate based on available resources. This approach achieves improved accuracy, lower communication and computation costs, and greater resilience against poisoning attacks.

Nikić et al. [42] explored the development of a digit recognition system for smart meters utilizing narrowband IoT and FL. Their proposed solution incorporates customized convolutional neural networks (CNNs) optimized for resource-constrained hardware, enabling efficient digit recognition in smart metering applications.

Colombo et al. [43] introduced TIFeD, a novel approach for training ML models on devices with constrained resources. This algorithm leverages integer arithmetic and direct feedback alignment (DFA) to overcome constraints. The results highlight the method's viability in severe memory and processing limitations.

Finally, Rimoli et al. [44] addressed the vulnerability of FL in MCU devices against attacks, such as model poisoning and data theft. The proposed solution combines FL with blockchain to protect model integrity and ensure data privacy.

Current research in TinyFL (*RQ1*) focuses on optimizing the performance of FL for resource-constrained IoT devices. Key research areas include memory and energy optimization, communication efficiency, security and privacy, model adaptation, heterogeneity, and real-world applications. These studies aim to enhance FL systems' efficiency, adaptability, and privacy in environments constrained by computational, memory, and energy limitations.

B. EXPLORING THE CONTEXTS OF TINYFL APPLICATIONS

This section explores the domains in which the selected studies researched TinyFL, addressing *RQ2*. By identifying these domains, we aim to understand the practical applications and contextual relevance of TinyFL and highlight the diversity and focus areas of current research efforts.

Muhammed and Ibraheem [11] aimed to detect anomalies and conducted digital forensic analysis focusing on data privacy in the context of low-power sensor networks. The proposal suits decentralized edge IoT environments where privacy and security are essential.

Giménez et al. [13], [23], [33] conducted research in scenarios involving low-power devices using LoRa for keyword recognition applications, where devices can train locally and collaborate to improve the model, such as in remote or low connectivity environments.

The focus on generating tiny, specialized, and personalized models is explored in [22], [24], and [27] in general IoT scenarios. The main goal is to reduce the size and complexity of ML models so that they can be executed on devices with memory and computational capacity constraints while maintaining good performance.

In this direction, Akubathini et al. [37] evaluated different ML optimization algorithms and techniques for resource-constrained IIoT. The main focus is industrial automation applications, where efficiency and data privacy are essential. It explores the use of FL and model compression to address resource-constrained challenges in these devices.

Privacy and security preservation are addressed by Mpofu et al. [41] in the context of urban agriculture and by Hidayat et al. [35], [40] in health applications and personal data monitoring. Along similar lines, Wu et al. [34] discussed reducing computational costs, while Zheng et al. [8] studied a framework for managing trust in FL scenarios involving UAV systems.

Application scenarios involving devices with minimal resources, scarce data availability, where computational and energy efficiency are critical, and lightweight ML models are required are addressed in [6], [28], [29], [30], [32], and [39].

Finally, Mwawado et al. [39] focused on using FL to optimize soil-based crop recommendations using Raspberry Pi edge computing nodes. The application scenario is smart

TABLE 5. Aggregation Algorithm

Aggregation Algorithm	Studies
FedAvg	[13], [25], [26], [33], [35]–[38], [41], [42], [44]
Custom Aggregation or Compression	[24], [27], [34], [40]
Custom FedAvg	[39]
Gossip Learning	[10], [22], [32]
Grey Wolf Optimization	[7]
Alpha-FedAvg	[11]
TFL-BC	[6]
TinyFL	[29]
TinyReptile/TinyMetaFed	[30], [31]
Not specified	[8], [9], [28]

agriculture, where data from soil sensors are utilized to enhance crop production.

Studies on TinyFL have explored applications across various domains (*RQ2*), including decentralized IoT networks, keyword recognition and remote environments, industrial automation and IIoT, smart agriculture, health and personal monitoring, smart cities and urban farming, UAV systems, resource-constrained devices, and smart metering. These studies demonstrate the broad adaptability of TinyFL in scenarios with resource, communication, and privacy constraints.

Although many FL studies advance research in the area, there is a clear need for more comprehensive assessments in scenarios that simulate real-world conditions. This includes addressing device heterogeneity, communication instability, environmental variations, model security, privacy techniques, decentralized architectures, compression methods, and more robust communication protocols. Research needs to consider these challenges so that FL can be implemented effectively in various practical applications.

The following sections provide the foundation for *RQ3*. We seek to comprehensively understand the related topics and their implications by addressing specific aspects of this research question.

C. MODEL AGGREGATION STRATEGIES AND METHODS

The selected studies attest to the execution of local training, that is, on the device itself, of ML models and the sharing of weights or gradients to perform FL. Thus, local data from devices are protected during the aggregation process, although there are techniques that prevent attacks from obtaining ML models.

Since training is performed locally, the next step is executing the aggregation process. When a given aggregator node receives the models from the various devices, it performs the aggregation according to established criteria and strategies, such as selecting clients that participated in the process, frequency, and aggregation strategy, and shares the aggregated model with the other devices in the network.

Table 5 summarizes the aggregation algorithms found in our research.

FedAvg is one of the most common aggregation algorithms in FL. It involves training models locally on client devices and sending the model parameters (weights and biases) to a central server, which averages these parameters to create a new global model. This global model is returned to the clients for the next training cycle [45].

Alpha-FedAvg [11] is an approach that dynamically adjusts the learning rate for each node by leveraging Lipschitz and smoothness parameters. This method modifies the learning rate based on the local characteristics of each device, aiming to achieve more efficient convergence. Despite these adjustments, the model aggregation process remains based on the traditional FedAvg technique.

Gossip learning is a decentralized method in which devices share model information with other randomly selected devices after each round of local training. It reduces communication costs but can lead to slower convergence [10], [22], [32].

These methods, custom aggregation methods, and compression deviate from the standard FedAvg, often focusing on compression to reduce communication costs or customization to accommodate non-IID data.

In the TIFeD algorithm, each hidden layer is trained independently, and models are aggregated after each mini-batch, buffer, or training iteration, adapting to the limitations of resource-constrained devices.

TinyReptile applies meta-learning techniques to IoT devices, sending model updates sequentially and focusing on rapidly adapting to new tasks.

TFL-BC is an FL algorithm focused on Bayesian classifiers. It aggregates local statistics to achieve efficiency on resource-constrained devices.

With TinyFL, only the latest updated layers of the models are sent to the server, significantly reducing communication costs. The updated models are then aggregated on a central server.

Finally, the grey wolf optimization genetic algorithm optimizes model parameters before aggregation, resulting in more accurate and efficient models suitable for embedded devices.

The studies reviewed highlight various model aggregation strategies in addressing *RQ3*, which explores the architectural and methodological choices in TinyFL implementations. Traditional methods, such as FedAvg, are widely utilized, alongside adaptations, such as Alpha-FedAvg and gossip learning, which aim to optimize communication and resource usage. We can identify alternative approaches involving customized aggregation or compression techniques and innovative strategies, such as TinyFL, TinyReptile, and grey wolf optimization. These methods tackle key challenges in FL, such as managing non-IID data distributions, minimizing communication overhead, and optimizing limited resources on constrained devices.

D. ML METHODS

Fig. 4 illustrates the distribution of ML and AI methods or algorithms used in our selected studies. The methods are grouped into four primary categories based on their frequency of occurrence.

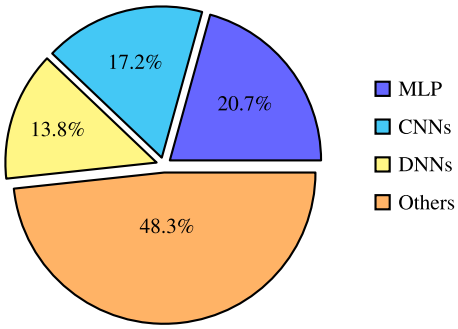


FIGURE 4. Distribution of ML/AI methods/algorithms.

Multilayer perceptron (MLP)—Feedforward (20.7%) represents the use of fully connected feedforward neural networks (FCNNs), which are commonly applied in structured data analysis and classification tasks.

Mpofu et al. [41] used an MLP-based model, an FCNN, for anomaly detection in water pumps in a federated IoT platform. Giménez et al. [13] employed an MLP, three-layer FCNN for keyword spotting (detecting keywords). Wulfert et al. [29] utilized NNs for gesture recognition. Several other applications of MLPs are discussed in the works of [10], [23], and [43].

CNNs (17.2%) highlight the application of CNNs, which are widely utilized for image processing, computer vision, and tasks requiring spatial data interpretation.

The application of CNNs for image classification is demonstrated by Aatab and Freitag [26]. Nikić et al. [42] employed a lightweight and customized CNN for digit recognition in smart meter readings. Other examples of CNN applications can be found in [22] and [27].

DNNs (13.8%) include general-purpose deep learning architectures designed for diverse applications requiring complex data analysis and high model capacity.

DNNs are extensively detailed in several works. For instance, Wu et al. [34] explored their applicability in structured data analysis, while Mwawado et al. [39] examined their use in resource-constrained environments. Hidayat et al. [40] provided insights into their role in solving complex optimization problems, and Cheng et al. [9] discussed advancements in their architectures for real-time processing.

Others (48.3%) are a collection of less prevalent methods, including recurrent neural networks (RNNs), meta-learning approaches, optimization techniques, and lightweight models, such as TinyML.

In addition to commonly used approaches, such as MLPs, CNNs, and DNNs, several other methods have been explored in the literature for various applications. Examples include Bayesian classifiers [6], FastGRNN [37], and grey wolf optimization (GWO) [7], which cater to specific use cases. Meta-learning approaches [30], [32], and RNNs [11] are also notable for their effectiveness in tasks involving time-series data. Linear regression (LR) [25] and multinomial logistic regression [8] provide simpler yet powerful solutions for structured data. Furthermore, hybrid methods, such as RNNs and SplitNNs [36], transfer learning (TL) [28], continuous-time NNs (CT-NNs) [24], and lightweight models, such as TinyML [38], showcase the diversity and adaptability of ML methods tailored to different domains and constraints.

To further answer *RQ3* regarding the methodologies in TinyFL, the studies demonstrate diverse ML models adapted for constrained environments. Frequently employed techniques include MLPs for structured data analysis and anomaly detection, CNNs for spatial and image-based tasks, and DNNs for more complex learning scenarios. In addition, specialized methods, such as Bayesian classifiers, meta-learning algorithms, and hybrid models, such as TinyReptile and SplitNN, highlight the adaptability of TinyFL solutions to the limitations and heterogeneity of resource-constrained devices.

E. STRATEGIES FOR RESOURCE-CONSTRAINED DEVICES IN TINYFL

The selected studies mention the devices used in their experiments but do not provide detailed technical specifications. Based on these devices and the experimental results reported by Ficco et al. [12], we present Table 6 that lists several devices commonly used in resource-constrained IoT environments focused on TinyML and TinyFL applications—*RQ4*. The devices are compared based on cost, power consumption, RAM capacity, architecture, number of cores, processing frequency, storage, and additional features, such as graphics processing unit (GPU) or floating point unit (FPU) support.

These features are essential for understanding how suitable each device is for different ML tasks, particularly when addressing power and memory constraints. The learning method column lists the learning methods implemented in each device, highlighting techniques, such as CNN, RNN, MLP, and LR. The table also references studies using these devices, offering practical context on their real-world applications.

We can observe that devices with lower costs and computational resources, such as the ESP32 DevKitC and the Arduino Nano BLE 3, can execute lightweight models, such as MLP, TL, CT-NN, LR, and CNN.

In addition to hardware, it is essential to consider the specific software requirements and appropriate tools to ensure the functionality of TinyFL applications.

The FL paradigm has evolved with the development of several dedicated tools and frameworks, each incorporating a variety of FL algorithms. These tools strategically facilitate the seamless application of FL in practical, real-world scenarios. Ray [14] and Dutta and Bharali [15] described in Table 7 several software and libraries used in configuring the Tiny environment. Building on this, we have expanded the table by including additional frameworks, programming languages, and integrated development environment (IDE) identified from the selected articles. This enhancement provides a comprehensive overview of the tools and technologies configuring the TinyML environment.

TABLE 6. Characteristics of Devices

Device	Cost (US$)	Supply (A)	RAM (MB)	Arch.	Cores	Freq. (MHz)	Storage	Add.	Studies	Learning Method
Arduino Nano BLE 33	25	0.5	0.256	32-bit	1	64	1 MB	-	[23], [24], [25], [28], [31], [43]	LR, MLP, TL, CT-NN
Arduino Portenta H7	120	0.6	8.0	32-bit	2	480	16 MB	GPU	[13], [26], [33], [44]	MLP, NN, CNN
Adafruit EdgeBadge	36	1.0	0.192	32-bit	1	120	512 KB	-		
ESP32 DevKitC	10	0.5	0.512	32-bit	2	240	4 MB	-	[10], [11], [24], [42], [44]	MLP, RNN, CNN, CT-NN
Raspberry Pi Pico W	7	0.5	0.256	32-bit	2	133	2 MB	-		
STM32F746	15	0.5	0.5	32-bit	1	216	1 MB	-	[37]	FastGRNN
Sipeed Maix Bit	45	1.0	8.0	64-bit	2	600	16 MB	FPU		
SparkFun Edge	17	0.5	0.384	32-bit	1	96	1 MB	FPU		
Banana Pi M2 Zero	20	2.0	512	32-bit	4	1200	8 GB	GPU		
Orange Pi Zero 3	35	2.0	1000	64-bit	4	1500	16 GB	GPU		
Raspberry Pi Zero W	15	1.2	512	32-bit	1	1000	16 GB	-		
Raspberry Pi Zero 2 W	20	1.2	512	64-bit	4	1000	16 GB	-		
Raspberry Pi 3 Model B	35	2.5	1000	64-bit	4	1200	16 GB	-	[30], [31], [36]	RNN, SplitNN, Meta-Learning
Raspberry Pi 4 model B	35	3.0	1000	64-bit	4	1800	32 GB	-	[32], [34], [35], [39], [40]	Meta-Learning, DNN, NN
Raspberry Pi 5	56	5.0	4000	64-bit	4	2400	64 GB	-		
NVIDIA Jetson Nano	99	5.0	4000	64-bit	4	1430	16 GB	GPU		

TABLE 7. Programming Languages, Software Frameworks, and IDEs

Language / Framework / IDE	Developer	Specialization	Studies
TensorFlow Lite Micro (TFLM)	Google	Open source. Flexible interpreter-based solution. The developer can modify the library code. Designed for a wide range of MCUs.	[10], [11], [13], [25], [29], [37]–[39]
Embedded Learning Library (ELL)	Microsoft	Open source library. Cross-compiler tool chain similar to TFLM.	
Graph Lowering (GLOW)	Research Group	Open source compiler. Uses ahead-of-time compilation for both floating-point and quantized arithmetic.	
STM32 X-CUBE.AI	STMicroelectronics	Optimized code for STM-32 series MCUs. It supports FP32 and quantized models and has built-in optimization.	
TensorFlow	Google	Compiles TensorFlow graphs into C++ code. Quantization and optimization are unavailable.	[32], [34]–[36]
TinyEngine	Research Group	Inference engine for MCUs. Compile-based method. Removes memory overhead.	
TVM	Research Group	Open source DL compiler. Offers optimized models for any hardware.	
uTensor	uTensor are specifically designed for ARM.	Converts TF model into C++.	
Edge Impulse	Edge Impulse, Inc.	Focused on end-to-end TinyML deployment. Provides tools for data collection, model training, and deployment. Supports multiple MCUs and architectures.	
NanoEdge	STMicroelectronics	Specializes in anomaly detection and classification. Provides pre-compiled binaries optimized for STM32 devices. No coding is required for model development.	
PyTorch Mobile	Meta Platforms, Inc.	Mobile and embedded inference with PyTorch. Supports Android and iOS devices. Focuses on optimizing models for mobile deployment.	
Python	Python Software Foundation	General-purpose programming language widely used for data preprocessing, model training, and TinyML prototyping.	[11], [23], [32]–[36], [39]
C/C++	Various Open Source Developers	Low-level programming languages used for optimized performance on MCUs in TinyML frameworks.	[10], [11], [13], [23], [25], [29]
IDE Arduino	Arduino	Integrated development environment for programming and deploying code on microcontrollers, widely used in TinyML projects.	[10], [11], [13], [23], [25], [29]
Keras	Google	High-level API for TensorFlow, allowing easy model creation and training. Often used in TinyML model development.	[11]
Scapy	Philippe Biondi	Python-based tool for network packet manipulation sometimes adapted for IoT communication in TinyML.	[11]
Spyder	Spyder Project Developers	Python IDE optimized for scientific computing and are often used in TinyML prototyping and data analysis.	[11]

In addressing *RQ4*, which examines the devices used in TinyFL implementations and strategies to overcome resource constraints, the studies highlight using various resource-constrained devices, such as ESP32, Arduino Nano BLE 33, and Raspberry Pi. These devices are selected for their low cost, energy efficiency, and compatibility with lightweight ML frameworks. Strategies to address limitations, such as memory, processing power, and energy consumption, include techniques, such as model compression, pruning, quantization, and feature extraction. Tools, such as TensorFlow Lite

Micro and STM32 X-CUBE.AI, optimize performance on these devices, ensuring feasibility for on-device learning in real-world TinyFL applications.

F. COMMUNICATION PROCESS

Studies approach FL communication process from diverse perspectives, exploring different techniques to optimize efficiency, protect privacy, and deal with the challenges of resource-limited environments and unstable networks.

Wulfert et al. [10] showed the efficiency of the gossip learning method by reducing communication and maintaining model accuracy. The amount of data exchanged between nodes was reduced by 57% compared to the segmented gossip method. Adopting quantization and learning rate adjustment resulted in a 19% increase in model accuracy. Furthermore, the proposal reduced the time needed to reach a suitable model. The evaluation showed an increase in data delivery rate of 2% when using 16-bit transfer instead of 32-bit and an increase of up to 12% when using 8-bit transfer. In the proposed method, devices exchange models locally instead of using a central server, reducing communication costs. In addition to modifying the Gossip protocol, the authors used quantization to reduce the size of the models and the AdaStair and AdaLoss methods to dynamically adjust the learning rate, improve convergence, and reduce the need for frequent communication.

Hidayat et al. [35] focus on reducing communication customs by applying adaptive compression and differential privacy. They use Pruning compression based on compressive sensing to reduce models' size before sending them to the server. Compression varies depending on device capacity (CPU, memory), ensuring that devices with fewer resources transmit smaller models. Compared to traditional methods, the total communication cost of the proposed method showed a reduction of 89% when compared to DP-FedAvg and 62% compared to DP-FedOpt.

Cheng et al. [9] used the sparse update mechanism technique to train and update the last I layers of the NN, reducing the communication cost. Furthermore, the devices only send the updated parameters instead of the complete model, and the algorithms manage packet losses in the communication channel. Compared to FedAvg, which updates the complete model and sends 476 KB, TinyFL sends 467 KB when the last 30 layers are updated and 347 KB when the last ten layers are updated. Regarding memory, FedAvg occupies 1400 KB when updating all layers, while TinyFL reduces this value to 771 KB when updating the last 30 layers and 403 KB when updating only the last ten layers. On the CIFAR-10 dataset, when updating only the last ten layers, the final accuracy was around 80% , while FedAvg reached 85% .

The exploration of reliable communication and energy efficiency in FL environments with LoRa mesh networks, a LoRa and low-power communication protocol, was demonstrated by Giménez et al. [13]. The authors implemented a peer-to-peer (P2P) protocol to exchange models between nodes, and the response time varied from 77 to 155 ms depending on the distance between them. The communication approach consumed just 100 mW, 33% less than centralized methods.

G. SECURITY AND PRIVACY MEASURES IN TINYFL RESEARCH

The studies presented different strategies to ensure security and privacy in FL, with approaches ranging from encryption and differential privacy to defense against model poisoning attacks. Table 8 summarizes main security and privacy measures.

Muhammed and Ibraheem [11] developed the AlphaFedAvg algorithm, introducing a dynamic parameter adjustment mechanism to enhance privacy and security in FL. The study included generating detailed forensic reports to monitor attack attempts and identify suspicious patterns in device communication. To test the system's resilience, 14 types of cyberattacks were simulated, including distributed denial of service (DDoS), ransomware, SQL injection, and MITM, using tools, such as Kali Linux, Tshark, and Scapy. The results indicated that the proposed approach efficiently mitigated attacks, protecting data privacy and ensuring the integrity of trained models.

Zheng et al. [8] introduced a blockchain-enabled FL model for UAV networks. Using blockchain allowed for more secure and reliable management of trained models, ensuring only authorized participants could contribute to FL. Researchers simulated model poisoning and parameter manipulation attacks to test the system. Malicious agents attempted to modify the models to compromise training. The results showed that integrating blockchain technology mitigated the attacks, enhancing robustness against external manipulations.

Hidayat et al. [40] proposed a resource-adaptive compression method combined with differential privacy to reduce communication overhead in FL. The model dynamically adjusted the gradient compression level based on the available resources of each device, enabling efficient training even on computationally constrained devices. Researchers simulated model poisoning attacks and modified data labels to evaluate the security of the approach. The results showed that the model successfully preserved privacy and resisted manipulation without significantly compromising learning accuracy.

H. IDENTIFIED GAPS AND CHALLENGES IN TINYFL RESEARCH

TinyFL is an emerging paradigm that aims to solve problems of latency, computational resource constraints, security, and privacy, for example. The selected studies reveal significant gaps in the current state of research, which are characteristic of emerging topics.

The first challenge encountered is the lack of implementation details of the presented solutions [10], including the lack of information on the security measures adopted, superficial description of the simulation of network attacks, and lack of information on the dataset used [11].

Another problem encountered is the scalability and evaluation of the measurements [13] since the experiments are

TABLE 8. Security and Privacy Measures

Study	Measures	Objective	Dataset	Attacks
[10]	Gossip protocol with quantization	Reduce reliance on a central server and protect trained models	MNIST and GR dataset (muscular sensor readings)	Focus on robustness against communication failures, no explicit attack simulation
[11]	Alpha algorithm, forensic analysis reports	Detect and mitigate cyberattacks while maintaining privacy	Edge-IIoTset and real network packets	Simulated 14 types of attacks (DDoS, ransomware, SQL injection, MITM) using Kali Linux, Tshark, and Scapy
[22]	Differential privacy, gradient compression	Protect user data without compromising training efficiency	Images collected from Raspberry Pi 4 and Android smartphones	Mentioned defense against Byzantine attacks but did not specify simulations
[25]	FedAvg parameter aggregation	Maintain data security on devices and prevent exposure of medical records	Hospital records from the US	No attack simulations performed, only parameter aggregation for data protection
[34]	AES encryption, differential privacy, digital signatures	Protect against inference attacks and information leakage	Not specified, simulated IoT scenarios	Evaluated impact of communication loss on model performance
[8]	Blockchain-enabled FL	Enhance trust and security in UAV networks	Simulated UAV data network	Simulated poisoning attacks and parameter tampering
[40]	Resource-Adaptive Compression with Differential Privacy	Reduce communication overhead while preserving privacy	MNIST and HAR datasets	Simulated poisoning attacks by modifying labels to test resilience
[38]	Attack mitigation, encryption, digital signatures	Prevent model poisoning and secure communication in FL	IoT sensor networks dataset	Simulated model poisoning attacks with anomaly detection techniques
[44]	Blockchain-based trust management	Improve security and robustness in TinyFL networks	Simulated UAV and IoT networks	Simulated model poisoning and parameter manipulation attacks
[31]	Federated meta-learning with Reptile	Enable fast adaptation in TinyFL while maintaining privacy	CIFAR-10 and TinyML datasets	No explicit security attacks simulated, but focused on privacy-aware model updates

performed with a specific set or reduced number of devices [24].

In addition to the scalability problem, details about the communication process or the data transfer rate are sometimes missing, as in [25].

Regarding FL, there is a lack of greater detail on selecting clients [22] for the aggregation process. This problem has been frequently studied in general FL contexts. Finally, there is the challenge of balancing the efficiency and effectiveness of ML models, given the limitations of models that can be applied on devices with resource constraints and the limitations of the local database itself.

The authors point out some perspectives for the future of TinyFL research, such as improving scalability, exploring devices with more severe resource restrictions, improving model security, exploring energy and computational efficiency strategies, investigating the impact of different client selection criteria, evaluating the performance of solutions in more extensive databases and, mainly, investigating the performance of solutions in real-world applications.

This section addresses *RQ5* by highlighting the main gaps and challenges revealed through the studies analyzed in our research, such as the lack of detailed implementation information, including insufficient data on security measures and datasets used, most of the experiments are conducted with a limited number of devices, and communication processes are often not fully detailed, and balancing efficiency and effectiveness in ML models on resource-constrained devices is an ongoing difficulty. Section VI extends this discussion, further exploring these and other unresolved issues while identifying promising directions for advancing TinyFL research.

VI. OPEN CHALLENGES

Applying FL in environments with resource-constrained devices introduces many open challenges. These challenges stem from the unique requirements of distributed systems, device heterogeneity, and the constraints of edge computing environments. In the following, we outline the key open challenges.

A. COMMUNICATION EFFICIENCY

Communication remains a significant bottleneck in FL, especially in centralized configurations, where communication overhead can consume over 60% of the total process time [9], [10], [22]. Optimizing communication through sparsification techniques, layerwise updates, or alternative aggregation mechanisms is critical to enabling FL in networks with limited bandwidth and high latency.

B. SCALABILITY

Adapting FL to large-scale ad-hoc networks, such as those with hundreds of microcontrollers, is challenging. The data overhead required for network maintenance and model synchronization limits scalability [9], [10], [11]. Novel approaches to decentralized FL and hierarchical aggregation are needed to address these issues.

C. HETEROGENEITY OF NETWORKS, DATA, AND MODELS

FL systems frequently encounter heterogeneity in both data and model architectures. Data non-IID distributions across clients hinder global model generalization, requiring techniques, such as clustering-based FL or personalized FL [8], [22]. Model heterogeneity remains an open challenge, as aggregating models with different architectures, layers, or parameters demands innovative approaches, such as knowledge distillation, where models exchange outputs (logits) instead of weights, allowing compatibility despite architectural differences [13], [30]. FL models must be adapted to IoT devices' different processing capabilities, power constraints, and communication instability [38].

D. RESOURCE CONSTRAINTS

Devices with limited memory (e.g., SRAM of 256 kB) and computational capacity pose significant challenges to deploying FL. Efficient resource utilization strategies, including model compression, pruning, and feature extraction, are crucial to making FL feasible in such environments [7], [23], [38].

E. PRIVACY AND TRUST MANAGEMENT

Ensuring the integrity and trustworthiness of FL models is critical, especially in applications involving sensitive data. Blockchain-based verification mechanisms and robust anomaly detection systems are needed to prevent model contamination and ensure reliable aggregation [8], [11]. While homomorphic encryption and advanced differential privacy offer improved security, they present implementation challenges on resource-constrained devices. The challenge lies in applying these techniques to optimize power consumption and processing efficiency while maintaining high levels of privacy without compromising performance [38].

F. EFFICIENT MODEL AGGREGATION

The aggregation of models in FL must balance accuracy, computational cost, and resource constraints. Developing adaptive aggregation strategies, such as geometric or personalized aggregation, is necessary for heterogeneous systems to account for varying hardware capabilities and data distributions [9], [25].

G. TRADEOFFS BETWEEN LOCAL AND GLOBAL TRAINING

Determining the optimal number of local training rounds before global aggregation is a persistent challenge. This tradeoff directly affects convergence rates, communication efficiency, and energy consumption, requiring context-aware strategies to strike the right balance [7], [13].

H. INFERENCE IN RESOURCE-CONSTRAINED DEVICES

Deploying FL models for continuous inference in constrained environments, such as IoT devices, remains an open problem [23], [26]. Techniques to support real-time inference with low latency and efficient resource utilization are essential for practical deployments.

I. ON-DEVICE TRAINING

Despite advances in FL, directly training ML models on resource-constrained devices remains challenging. Most solutions focus solely on on-device inference or model adaptation, leaving training to external cloud services [43].

J. META-LEARNING AND MODEL ADAPTATION

Teaching NNs to quickly adapt to new tasks with limited labeled data through meta-learning presents an exciting opportunity [27], [30]. However, achieving this in federated settings, especially with heterogeneous devices and data, is still challenging [27], [28].

K. DATA STORAGE ON DEVICES

An essential challenge in TinyFL is the lack of strategies and implementations for efficient local data storage on resource-constrained devices. While local training is a key component of FL, the studies reviewed often omit details about how data are stored and managed on devices with limited memory.

L. ADVANCED FL TECHNIQUES FOR TINYFL

Before exploring advanced techniques in TinyFL, it is necessary to understand how data are distributed among the participating devices.

Data partitioning refers to how data are distributed across the sample and feature spaces, allowing FL systems to be categorized into three main approaches: horizontal, vertical, and hybrid [46].

1) Horizontal FL occurs when different devices have the same feature space but little intersection in their sample space. This is typical in IoT sensor networks where distributed sensors collect the same type of data in different locations.
2) Vertical FL applies when different devices share the same or similar sample space, but the collected data consist of different features. A common example is a scenario where one set of sensors measures temperature while another measures humidity, both monitoring the same environment.
3) Hybrid FL combines horizontal and vertical FL elements, where the data partitioning among devices includes aspects of both. This scenario is common in heterogeneous IoT networks, where some sensors collect similar data while others gather entirely different attributes.

These different data distribution strategies directly impact the techniques used in TinyFL, influencing decisions regarding model aggregation, personalized learning, and communication efficiency. Several advanced techniques have been explored to improve the efficiency and adaptability of FL on resource-constrained devices. This work extends the review of TinyFL by incorporating three emerging approaches: federated meta-learning, FTL, and distributed (serverless) FL.

Federated meta-learning aims to improve the generalization of models to new devices and data-poor scenarios. This approach allows models to quickly learn common patterns and adapt to different local data distributions. Work, such as TinyReptile [30], combines FL with meta-learning to find efficient weight initializations, enabling fast training on IoT devices with limited data. Meta-learning can also be applied to tune hyperparameters, automatically reducing the impact of non-IID distributions.

In IoT applications, different devices may have data that do not share the same structure or statistical distribution. FTL allows pretrained models to be fine-tuned for new devices, improving customization without needing training from scratch. Works, such as TinyFedTL [28], explore this

approach to enable the customization of federated models on embedded devices. FTL can be applied in scenarios where sensors from different manufacturers collect similar data but with specific hardware characteristics that affect data distribution.

Dependence on a central server can be a critical limitation in highly distributed embedded systems. Distributed (server-less) FL eliminates the need for a central aggregator, allowing devices to exchange models directly with each other. Techniques, such as gossip learning [10] and decentralized collaborative learning [32], have been proposed to reduce the computational load on a single node and increase network resilience. This model can be helpful for distributed IoT sensor networks in environments where connectivity to central servers is intermittent or nonexistent.

In summary, these open challenges highlight the need for continued innovation in communication protocols, model aggregation techniques, and adaptive learning methods to advance the state of FL in resource-constrained environments.

VII. CONCLUSION AND FUTURE WORKS

The TinyFL paradigm has garnered considerable attention from researchers seeking to implement FL at the edge. In this article, we contribute with a systematic review of the literature in TinyFL.

The 28 studies analyzed allowed us to map the current state of research in TinyFL based on each work's focus, application domains, architecture, and methodology used to implement FL in tiny devices, which devices are being or can be used, and the challenges and future work in the area.

Despite the challenges inherent to the paradigm, such as resource restrictions, TinyFL has the potential to explore varied domains of intelligent applications at the edge of the network, proposing innovative solutions in different fields and providing opportunities for new studies. Thus, it is becoming an up-and-coming area.

Based on the analyzed studies, an excellent future research opportunity is to investigate other methods of transferring learning, such as knowledge distillation. This is because existing studies always focus on a single set of devices with the same configurations and running the same models with the same hyperparameters. This limitation makes it difficult for heterogeneous devices and models to participate in an IoT system.

Another perspective for future work is to evaluate the process of storing data locally, that is, on the device itself. We infer that it is necessary to maintain a local database to perform local training, but the works studied did not address this topic.

ACKNOWLEDGMENT

The authors would like to thank the CNPq, CAPES, and FAPESB organizations for supporting the Graduate Program in Computer Science at the Federal University of Bahia.

REFERENCES

[1] A. Karras, A. Giannaros, C. Karras, K. C. Giotopoulos, D. Tsolis, and S. Sioutas, "Edge artificial intelligence in large-scale IoT systems, applications, and Big Data infrastructures," in *Proc. 2023 8th South-East Europe Des. Automat., Comput. Eng., Comput. Netw. Social Media Conf.*, 2023, pp. 1–8.

[2] S. Ni, Y. He, L. Chen, Y. Wang, and F. Yu, "A survey of edge computing resource allocation strategies based on federated learning," in *Proc. 2023 Int. Conf. Netw. Netw. Appl.*, Los Alamitos, CA, USA, Aug. 2023, pp. 116–121.

[3] P. Warden and D. Situnayake, *TinyML: Machine Learning With TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly, 2020.

[4] M. Shafique, T. Theocharides, V. J. Reddy, and B. Murmann, "TinyML: Current progress, research challenges, and future roadmap," in *Proc. 2021 58th ACM/IEEE Des. Autom. Conf.*, 2021, pp. 1303–1306.

[5] V. Tsoukas, A. Gkogkidis, and A. Kakarountas, "Internet of Things challenges and the emerging technology of TinyML," in *Proc. 2023 19th Int. Conf. Distrib. Comput. Smart Syst. Internet Things*, 2023, pp. 491–495.

[6] N. Xiong and S. Punnekkat, "Tiny federated learning with Bayesian classifiers," in *Proc. 2023 IEEE 32nd Int. Symp. Ind. Electron.*, 2023, pp. 1–6.

[7] S. Adhikary and S. Dutta, "FedTinyWolf—A memory efficient federated embedded learning mechanism," *IEEE Embedded Syst. Lett.*, vol. 16, no. 4, pp. 513–516, Dec. 2024.

[8] J. Zheng et al., "Trust management of tiny federated learning in internet of unmanned aerial vehicles," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21046–21060, Jun. 2024.

[9] H. Cheng, Q. Chen, Y. Liang, and G. Zhu, "TinyFL: A lightweight federated learning method with efficient memory-and-communication," in *Proc. 2023 IEEE Globecom Workshops*, 2023, pp. 608–613.

[10] L. Wulfert, N. Asadi, W.-Y. Chung, C. Wiede, and A. Grabmaier, "Adaptive decentralized federated gossip learning for resource-constrained IoT devices," in *Proc. 4th Int. Workshop Distrib. Mach. Learn.*, in Series DistributedML'23, New York, NY, USA, 2023, pp. 27–33.

[11] K. Muhammed and N. Ibraheem, "Alpha-FedAvg: Safeguarding privacy and enhancing forensic analysis in federated learning on edge devices," *Int. J. Comput. Digit. Syst.*, vol. 15, pp. 1391–1403, 2024.

[12] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, "Federated learning for IoT devices: Enhancing TinyML with on-board training," *Inf. Fusion*, vol. 104, 2024, Art. no. 102189.

[13] N. L. Giménez, J. M. Solé, and F. Freitag, "Embedded federated learning over a LoRa mesh network," *Pervasive Mobile Comput.*, vol. 93, 2023, Art. no. 101819.

[14] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 4, pp. 1595–1623, 2022.

[15] D. L. Dutta and S. Bharali, "TinyML meets IoT: A comprehensive survey," *Internet Things*, vol. 16, 2021, Art. no. 100461.

[16] F. Oliveira, D. G. Costa, F. Assis, and I. Silva, "Internet of Intelligent Things: A convergence of embedded systems, edge computing and machine learning," *Internet Things*, vol. 26, 2024, Art. no. 101153.

[17] N. Tekin, A. Aris, A. Acar, S. Uluagac, and V. C. Gungor, "A review of on-device machine learning for IoT: An energy perspective," *Ad Hoc Netw.*, vol. 153, 2024, Art. no. 103348.

[18] P. Qi, D. Chiaro, and F. Piccialli, "Small models, big impact: A review on the power of lightweight federated learning," *Future Gener. Comput. Syst.*, vol. 162, 2025, Art. no. 107484.

[19] E. Fragkou and D. Katsaros, "A joint survey in decentralized federated learning and TinyML: A brief introduction to swarm learning," *Future Internet*, vol. 16, 2024, Art. no. 413.

[20] Z. Stapic, E. G. López, A. G. Cabot, L. d. M. Ortega, and V. Strahonja, "Performing systematic literature review in software engineering," in *Proc. Central Eur. Conf. Inf. Intell. Syst.*, Varazdin, Croatia: Fac. Org. Inform., 2012, Art. no. 441.

[21] B. A. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University Joint Report, Tech. Rep. EBSE 2007-001, Jul. 2007.

[22] H. Huang, L. Zhang, C. Sun, R. Fang, X. Yuan, and D. Wu, "Distributed pruning towards tiny neural networks in federated learning," in *Proc. 2023 IEEE 43rd Int. Conf. Distrib. Comput. Syst.*, 2023, pp. 190–201.

[23] N. L. Giménez, M. M. Grau, R. P. Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with federated learning," *Electronics*, vol. 11, no. 4, 2022, Art. no. 573.
[24] Y. Dai and R. Tan, "FedCFC: On-device personalized federated learning with closed-form continuous-time neural networks," in *Proc. 2024 23rd ACM/IEEE Int. Conf. Inf. Process. Sensor Netw.*, 2024, pp. 14–26.
[25] S. Sriram, R. K. Hariharathmajan, A. Pradeep, and R. Karthi, "Federated learning on low-power Arduino Nano33 BLE sense to predict the length of stay using a linear regression model," *Procedia Comput. Sci.*, vol. 235, pp. 671–682, 2024.
[26] S. Aatab and F. Freitag, "Integrating a neural network library in embedded federated learning," in *Proc. 2023 IEEE 6th Int. Conf. Cloud Comput. Artif. Intell.: Technol. Appl.*, 2023, pp. 1–7.
[27] M. Das, G. Bagwe, M. Pan, X. Yuan, and L. Zhang, "Learning, tiny and huge: Heterogeneous model augmentation towards federated tiny learning," in *Proc. 2023 Int. Conf. Mach. Learn. Appl.*, 2023, pp. 90–97.
[28] K. Kopparapu, E. Lin, J. G. Breslin, and B. Sudharsan, "TinyFedTL: Federated transfer learning on ubiquitous tiny IoT devices," in *Proc. 2022 IEEE Int. Conf. Pervasive Comput. Commun. Workshops Other Affiliated Events*, 2022, pp. 79–81.
[29] L. Wulfert, C. Wiede, and A. Grabmaier, "TinyFL: On-device training, communication and aggregation on a microcontroller for federated learning," in *Proc. 2023 21st IEEE Interregional NEWCAS Conf.*, 2023, pp. 1–5.
[30] H. Ren, D. Anicic, and T. A. Runkler, "TinyReptile: TinyML with federated meta-learning," in *Proc. 2023 Int. Joint Conf. Neural Netw.*, 2023, pp. 1–9.
[31] H. Ren, X. Li, D. A. Anicic, and T. Runkler, "TinyMetaFed: Efficient federated meta-learning for TinyML," in *Machine Learning and Principles and Practice of Knowledge Discovery in Databases. ECML PKDD 2023. Communications in Computer and Information Science* , vol. 2136, R. Meo and F. Silvestri, Eds. Cham, Switzerland: Springer, 2025.
[32] T. A. Khoa, D.-V. Nguyen, M.-S. Dao, and K. Zettsu, "AOP: Towards adaptive offloading point approach in a federated learning framework for edge AI applications," in *Proc. 2023 IEEE 29th Int. Conf. Parallel Distrib. Syst.*, 2023, pp. 2846–2847.
[33] N. L. Giménez, F. Freitag, L. Navarro, and M. Selimi, "Demo: Edge federated learning over a LoRa mesh network," in *Proc. 2024 IEEE/ACM Symp. Edge Comput.*, 2024, pp. 510–511.
[34] J. Wu, Q. Xia, and Q. Li, "Efficient privacy-preserving federated learning for resource-constrained edge devices," in *Proc. 2021 17th Int. Conf. Mobility, Sens. Netw.*, 2021, pp. 191–198.
[35] M. A. Hidayat, Y. Nakamura, and Y. Arakawa, "Efficient and secure: Privacy-preserving federated learning for resource-constrained devices," in *Proc. 2023 24th IEEE Int. Conf. Mobile Data Manage.*, 2023, pp. 184–187.
[36] Y. Gao et al., "End-to-end evaluation of federated learning and split learning for Internet of Things," in *Proc. 2020 Int. Symp. Reliable Distrib. Syst.*, 2020, pp. 91–100.
[37] P. Akubathini, S. Chouksey, and H. S. Satheesh, "Evaluation of machine learning approaches for resource constrained IIoT devices," in *Proc. 2021 13th Int. Conf. Inf. Technol. Elect. Eng.*, 2021, pp. 74–79.
[38] W. Villegas-Ch, R. Gutierrez, A. M. Navarro, and A. Mera-Navarrete, "Optimizing federated learning on TinyML devices for privacy protection and energy efficiency in IoT networks," *IEEE Access*, vol. 12, pp. 174354–174370, 2024.
[39] R. Mwawado, M. Zennaro, J. Nsenga, and D. Hanyurwimfura, "Optimizing soil-based crop recommendations with federated learning on Raspberry Pi edge computing nodes," in *Proc. 2024 11th Int. Conf. Internet Things: Syst., Manage. Secur.*, 2024, pp. 82–89.
[40] M. A. Hidayat, Y. Nakamura, and Y. Arakawa, "Privacy-preserving federated learning with resource-adaptive compression for edge devices," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13180–13198, Apr. 2024.
[41] P. Mpofu, S. H. Kembo, M. Chimbwanda, S. Jacques, N. Chitiyo, and K. Zvarevashe, "A privacy-preserving federated learning architecture implementing data ownership and portability on edge end-points," *Int. J. Ind. Eng. Operations Manage.*, vol. 5, pp. 118–134, Apr. 2023.
[42] V. Nikić, D. Bortnik, M. Lukić, D. Vukobratović, and I. Mezei, "Lightweight digit recognition in smart metering system using narrowband Internet of Things and federated learning," *Future Internet*, vol. 16, no. 11, 2024, Art. no. 402.
[43] L. Colombo, A. Falcetta, and M. Roveri, "TIFeD: A tiny integer-based federated learning algorithm with direct feedback alignment," in *Proc. 3rd Int. Conf. AI-ML Syst.*, in Series AIMLSystems'23, New York, NY, USA: Assoc. Comput. Machinery, 2024, pp. 1–8.
[44] G. P. Rimoli, B. Boi, P. Fusco, C. Esposito, and M. Ficco, "Tiny federated learning with blockchain for privacy and security preservation of MCU-based IoT applications," in *Proc. 2024 6th Int. Conf. Blockchain Comput. Appl.*, 2024, pp. 124–131.
[45] S. I. Nanayakkara, S. R. Pokhrel, and G. Li, "Understanding global aggregation and optimization of federated learning," *Future Gener. Comput. Syst.*, vol. 159, pp. 114–133, 2024.
[46] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.



CLAUDIO JR. NASCIMENTO DA SILVA received the M.Sc. degree in computer science from the Federal University of Sergipe, São Cristóvão, Brazil, in 2017, and a degree in systems analysis and development from the Jorge Amado University Center (UNIJORGE - 2020), Salvador, Brazil, in 2020. He is currently working toward the Ph.D. degree with the Federal University of Bahia, Salvador.

He has experience in computer science, emphasizing information systems, software engineering, and strategic planning. He is currently is a participant in the Web, Internet, and Intelligent Systems Research Group (WISER). His research interests include Internet of Things, blockchain, fog computing, artificial intelligence, and Industrial Internet of Things.



CÁSSIO V. S. PRAZERES (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Sao Paulo, Sao Paulo, Brazil, in 2009.

Since 2010, he has been an Associate Professor in the Web/Internet area with the Federal University of Bahia (UFBA), Salvador, Brazil, where he leads research activities and projects, teaches undergraduate and postgraduate courses, and is an Advisor for Ph.D., M.Sc., and B.Sc. students. Also, at UFBA, he is a Co-founder and Leader of a laboratory and CNPq research group (WISER: Web, Internet, and Intelligent Systems Research Group). He has several publications in international conferences and journals. His research interests include IoT, microservices, fog/edge computing, and edge AI.

Dr. Prazeres is a Member of the Brazilian Computer Society; IEEE Communications Society; IEEE Computer Society Technical Committee on Services Computing; IEEE Smart Cities Technical Community; IEEE Internet of Things Technical Community; and W3C Web of Things Community Group.